MOBILE APP DEVELOPMENT ASSESSMENT 2

BeGrateful App Development Report

Matthew Cornwell

Introduction

BeGrateful is a mental health focused gratitude journalling app, prioritising easy access to small moments of gratitude each day. This goal is aided in part by minimal intrusion on the phone, and also by the lack of an overarching social media aspect.

The design choice to focus on a camera first design was rooted in the idea of "Mobile only moments" [1] from the lectures. This theory defines a mobile moment as a moment relevant to their immediate context, which in this case is trying to bring more mindfulness to our neurodivergent users. An industry leading study from Falaki et al [2] shows that "the median mobile interaction is 56 seconds", offering unique opportunities in comparison to a desktop application.

To assist in the ease of use necessary for the modern-day mobile moment, I split the app into 3 simple tabs with a navigation bar along the bottom implemented with tabs. When opening the app, the user lands on the capture screen, allowing them to capture "Moments" to be grateful for instantly. The other main page is the Reflect page, allowing users to view past memories stored in an offline SQLite database with relevant metadata. The final page is the settings page, providing the users with accessibility features such as the ability to pick a theme, change font size and delete all data from the database. The simple tab layout minimises cognitive load for the user and streamlines the path of least resistance to maximise mental health benefits.

I attempted to add a notification reminder setting like I had planned in Assessment 1, but ran into issues with my Expo Go version and Android notifications. The system threw an error which informed me I'd either have to deprecate or update to the dev build, so I omitted notifications in this build, the silver lining being that it remains an unintrusive non-social-media safe space for the user.

Development

Element 1: The creation of a Moment

The most important element and the core of the entire app is the “Moment”. The following lifecycle shows how a moment is created, how it’s modified, and how it persists in a CRUD database.

```
export const initDatabase = () => {
  db.execSync(`
    CREATE TABLE IF NOT EXISTS moments (
      id INTEGER PRIMARY KEY AUTOINCREMENT,
      image_uri TEXT NOT NULL,
      caption TEXT,
      location_label TEXT,
      weather_temp TEXT,
      date_created TEXT
    );
  `);
};
```

Gratitude journalling and mental health can be a very private subject for some people, leading to the choice to use an offline SQLite database running locally to store the potentially sensitive “Moments”.

The offline approach offers 2 main benefits: mental health and privacy. In recent years social media has been proved to increase social anxiety among emerging adults [3], so by going fully offline it turns the app into more of a journal or quiet moment for reflection, not something broadcasted to the world. The offline approach also ensures that user data never leaves the local storage of the device, eliminating cloud security risk. As a final bonus it also allows full functionality while roaming which is perfect for a mobile context.

To ensure stability and ease of use I created the database in its own database.tsx script. From a technical perspective SQLite was the right choice as Firebase would require cloud interaction, and AsyncStorage isn’t as flexible or robust. I initialised the table only “IF NOT EXISTS”, necessary for local storage to avoid crashes. I also created helper functions like getMoments and addMoment to be used in other scripts.

Once the database schema is built, we define a moment as a TypeScript interface to enforce uniform communication and a rigid base for the moment to be carried throughout the app. This ensures type safety across the app, and choosing mainly strings keeps the moment lightweight.

```
export interface Moment {
  id: number;
  image_uri: string;
  caption: string;
  location_label: string;
  weather_temp: string;
  date_created: string;
}
```

Once a user has taken a photo they’re presented with the ability to add a comment to the image and also add weather / location data if they choose. I designed the system to use the hardware’s integrated GPS, with the only thing required from the user being that permissions have to be accepted, allowing the user to remember the context of their moment at a later date.

Calling these APIs counts as a “blocking operation”, meaning that the entire UI would freeze while waiting for the calls to complete, which destroys responsiveness and user retention. To get around this I used asynchronous functions [4], allowing the time consuming calls to go on in the background while maintaining the user experience

Since GPS sensors can be unreliable some error handling is necessary. I added fallback logic by piping “Unknown”, which ensures that if the Meteo API call fails then the location is set to “Unknown” preventing it from throwing errors. In order for the app to continue functioning in the background this fetch implementation required an asynchronous function.

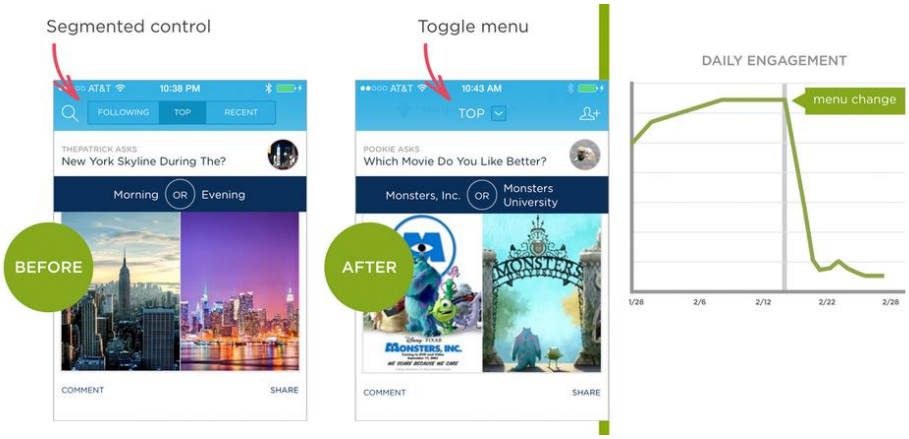
```
export const addMoment = (
  image: string,
  caption: string,
  location: string,
  weather: string
) => {
  db.runSync(
    "INSERT INTO moments (image_uri,
    image,
    caption,
    location,
    weather,
    new Date().toISOString()
  );
};
```

Upon completion of the moment creation, the data is inserted into the database via the addMoment function. This function executes the insert by using runSync, allowing it to maintain data integrity.

Element 2: Reflecting on moments

The other side of the coin is reflecting on the moments, arguably the more crucial of the two, allowing users to look back and see and interact with all the captured moments. On a technical level, this involves continued persistence, retrieval and accessibility / aesthetics.

To start the reflection, you have to first navigate from the Capture screen to the reflect screen. I decided to create a persistent bottom tab bar using the Expo Router, keeping the relevant screens inside a route group folder called “tabs”, to be displayed when the bar is clicked. Luke Wroblewski [5] wrote “obvious always wins” when it comes to navigation, so having all 3 main pages: Settings, Capture, Reflect as a constant bottom bar throughout all of the browsing creates a smoother user experience. Wroblewski also shows evidence that switching to a toggle menu instead of the segmented menu currently implemented dropped user retention by 90%.



The first step of reflection is the retrieval of data on both the gallery and reflect scripts. When the reflect screen is opened, the useFocusEffect listener runs setData(getMoments()) whenever the screen is brought into focus.

```
useFocusEffect(() => {
  setData(getMoments());
});
```

The array of moments is then fed into a FlatList with 3 columns, which renders each moment in a scrollable gallery and displayed in a grid. By importing the Dimensions API it allows us to get the width of the current device's window, then calculate the size with a simple algorithm that ensures everything stays proportional no matter the environment.

```
<FlatList
  data={data}
  keyExtractor={(item) => item.id.toString()}
  renderItem={renderItem}
  numColumns={3}
/>
```

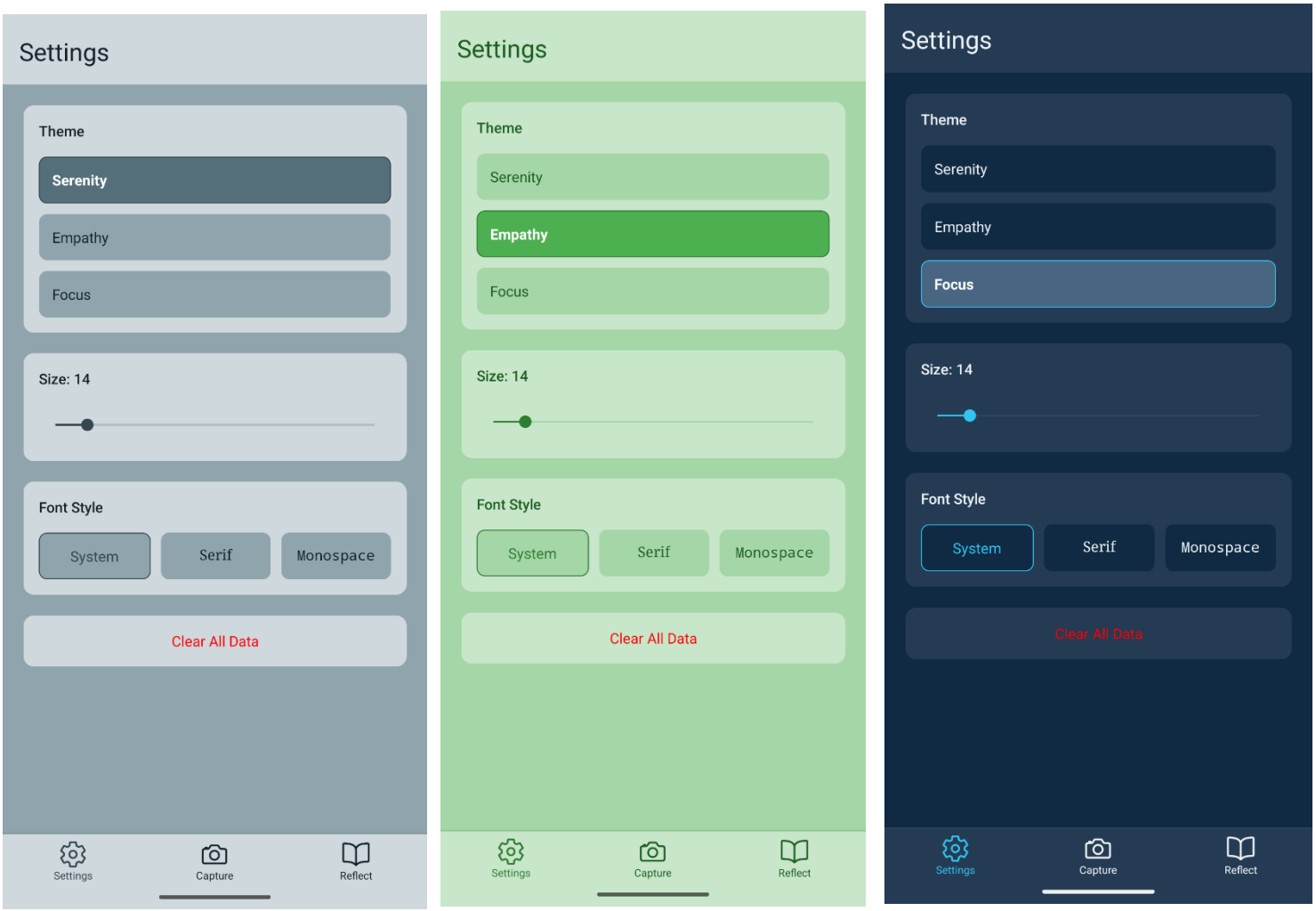
```
const width = Dimensions.get("window").width;
const screenPadding = 10;
const imageSize = (width - screenPadding * 2) / 3;
```

Once one of the rendered items in that FlatList is clicked, the app begins the transition to the single gallery view. The onPress() call triggers router.push, giving the unique item ID of the Moment that was selected. Once in this gallery view users can delete or save their moments using functions from the database script, as well as swipe across images.

```
const renderItem = ({ item }: { item: Moment }) => (
  <TouchableOpacity
    onPress={() =>
      router.push({ pathname: "/gallery", params: { initialId: item.id } })
    }
    style={styles.itemContainer}
  >
```

To support neurodivergent users with specific sensory, visual processing issues or dyslexia I chose to build a themes system into the app allowing users freedom of choice and customisation. This adheres to Nielsen's 7th Heuristic of User Control and Freedom [6] which

states “allow users to tailor frequent actions” to improve flexibility and efficiency. I created 3 preset calming themes so as to not overwhelm the user with full colour customisation.



Themes

To achieve homogeneity across all pages efficiently I utilised the React Context API as a form of state management. By defining a context provider all the screens could have a single source of data relating to the themes making it scalable and efficient. I found a balance of keeping all the static layout theming at the bottom in the stylesheet and then updating all theme properties inline via calls to the style’s palette.

```
<View style={[styles.card, { backgroundColor: colors.card }]}>
  <Text
    style={[
      styles.label,
      { color: colors.text, fontFamily: font, fontSize },
    ]}
  >
```

When an image is saved, to ensure privacy of reflection and the app as a whole permissions must be requested. Following standard practice by Google as quoted by Almuhimedi, “Google replaced install-time permission screens with just-in-time permission requests” [7], noting that users have an easier time understanding permissions requests when relevant to the current context. It also decidedly only asks for write permissions to save images to the camera roll, not any read permissions, ensuring minimal intrusion on the device.

```
const handleSave = async () => {
  const item = moments[currentIndex];
  if (!item) return;
    const { status } = await MediaLibrary.requestPermissionsAsync(true);
    if (status === "granted") {
      await MediaLibrary.saveToLibraryAsync(item.image_uri);
      Alert.alert("Saved");
    } else {
      Alert.alert("Permission required");
    }
};
```

Reflection

Reflecting on this project, as is done in the app, I find that the syntax is very similar to desktop web development platforms, but the most crucial distinction is the mobile context in which the software runs. Desktop environments are generally relatively static, but the dynamic nature of mobile leads to different design challenges. For example, having to utilise “useFocusEffect()” every time the gallery is loaded to refetch everything feels unique to React Native for me.

Implementing location and camera functionality is reminiscent of web dev again, but the context is completely different. The “mobile only moments” nature of the app allows users to capture gratitude at any moment during their day, effectively making a live journal, whereas the desktop is a static diary constrained to one location.

Budiu mentions the classic adage to “prioritise content over chrome” [8]. Mobile screens suffer from a small size and the single screen constraint, meaning that apps have to be designed to avoid user fatigue, fat finger errors, and sharp focus of each screen instead of the desktop counterpart of 10 tabs each with 40 buttons. I also put extra importance on the navigation bar being in the “thumb zone”, an area that heatmaps reveal the users are most naturally drawn to.

Having to ask for permissions with push notifications is another major difference between mobile and desktop design, with privacy being doubly important on a phone for the average user than on a PC.

References

1) Bachour, K. (2025). Mobile Application Development Week 4: Mobile Usability. (Accessed 12/01/2026)
2) Falaki, H., Mahajan, R., Kandula, S., Lymberopoulos, D., Govindan, R. and Estrin, D. (2010). Diversity in smartphone usage. Proceedings of the 8th international conference on Mobile systems, applications, and services - MobiSys '10. doi:https://doi.org/10.1145/1814433.1814453.
3) Vannucci, A., Flannery, K.M. and Ohannessian, C.M. (2017). Social media use and anxiety in emerging adults. Journal of Affective Disorders, [online] 207, pp.163–166. doi:https://doi.org/10.1016/j.jad.2016.08.040.
4) Bachour, K. (2025). Mobile Application Development Week 6: Geolocation and Asynchronous Functions. (Accessed 12/01/2026)
5) Wroblewski, L. (2015). LukeW | Obvious Always Wins. [online] Lukew.com. Available at: https://www.lukew.com/ff/entry.asp?1945 [Accessed 14 Jan. 2026].
6) Nielsen Norman Group. (2025). Usability Engineering : Book by Jakob Nielsen. [online] Available at: https://www.nngroup.com/books/usability-engineering/ [Accessed 14 Jan. 2026].
7) Almuhimedi, H. (2017). Helping Smartphone Users Manage their Privacy through Nudges. [online] Available at: http://reports-archive.adm.cs.cmu.edu/anon/isr2017/CMU-ISR-17-111.pdf [Accessed 14 Jan. 2026].
8) Raluca Budiu (2014). Maximize the Content-to-Chrome Ratio, Not the Amount of Content on Screen. [online] Nielsen Norman Group. Available at: https://www.nngroup.com/articles/content-chrome-ratio/ [Accessed 14 Jan. 2026].